

C# for High-Performance Numerical and Scientific Code

TYPES · CLASSES · ARRAYS · SPANS · REFS · MEMORY LAYOUT · HOT LOOPS · DETERMINISTIC APIS · C# 14 / .NET 10

A TYPICAL C# PROGRAM

Single-File Console Application

```
// File: Program.cs -- this is the entire program
using System.Globalization;

int n = 100;
double[] x = new double[n];
double[] y = new double[n];

for (int i = 0; i < n; i++)
    x[i] = Math.Sin(i * 0.1);

Scale(x, 2.0);
double d = Dot(x, x, n);
Console.WriteLine($"n = {n}, dot = {d:G10}");

// Static local functions:
static void Scale(Span<double> v, double a)
{
    for (int i = 0; i < v.Length; i++) v[i] *= a;
}

static double Dot(ReadOnlySpan<double> a,
    ReadOnlySpan<double> b, int n)
{
    double s = 0;
    for (int i = 0; i < n; i++) s += a[i] * b[i];
    return s;
}
```

What This Illustrates

- **Top-level statements:** no **class**, no **Main**. The file is the entry point.
- **Static local functions:** defined after calling code, in the same file.
- **Span parameters:** **Scale** takes a **Span<double>**—an array is implicitly convertible.
- **String interpolation:** **"\$" . . . {d:G10} "** for formatted output.

TO COMPILE AND RUN

```
// $ dotnet new console -n MyApp
// $ cd MyApp && dotnet run -c Release
```

No headers, no makefiles, no linking step.

PRIMITIVE TYPES, MATH, AND SYNTAX ESSENTIALS

Numeric and Primitive Types

Type	Description
int / long	signed 32 / 64-bit integer
float / double	32 / 64-bit IEEE 754
decimal	128-bit exact decimal
bool	true/false
byte, short	unsigned 8 / signed 16-bit
uint, ulong	unsigned 32/64-bit
nint, nuint	platform-native size
char	UTF-16 character
string	immutable character sequence
All numeric types are fixed-size. Complex numbers via <code>System.Numerics.Complex</code> .	

Control Flow Essentials

```
for (int i = 0; i < n; i++) { /* ... */ }
while (residual > tol) { /* ... */ }
do { /* ... */ } while (residual > tol);

if (x > 0) { /* ... */ }
else if (x == 0) { /* ... */ }
else { /* ... */ }
```

- `double y = (x > 0) ? x : -x; // ternary`
- Statements end with **;**. Blocks use **{ }**. Case-sensitive.
 - **break** exits a loop; **continue** skips to next iteration.
 - **for (;;)** is an infinite loop; exit with **break** or **return**.

Math via `System.Math`

Method	Description
<code>Math.Sin</code> , <code>Cos</code> , <code>Tan</code>	trigonometric
<code>Math.Asin</code> , <code>Acos</code> , <code>Atan</code>	inverse trig
<code>Math.Atan2</code> (y, x)	two-argument arctangent
<code>Math.Sqrt</code> (x)	square root
<code>Math.Pow</code> (a, b)	a^b
<code>Math.Exp</code> (x), <code>Log</code> (x)	e^x , $\ln x$
<code>Math.Abs</code> (x)	absolute value
<code>Math.Max</code> (a, b), <code>Min</code>	extrema
<code>Math.Ceiling</code> , <code>Floor</code> , <code>Round</code>	rounding
Math operates on double ; use MathF for float . No power operator; use Math.Pow or explicit multiplication.	
Special: <code>double.NaN</code> , <code>double.PositiveInfinity</code> , <code>double.Epsilon</code> . Test with <code>double.IsNaN(x)</code> .	

C# LANGUAGE FOUNDATIONS

Namespaces, using, and Program Entry

```
// File: Program.cs (top-level statements)
Console.WriteLine("Hello, World!");
```

Other files use file-scoped namespaces:

```
namespace MyFEM; // file-scoped (no braces)

using System.Numerics; // import namespace
using static Math; // import static members
using Vec = System.Numerics.Vector<double>; // alias
global using System.Globalization; // project-wide

public class Solver { /* ... */ }
```

- **using:** import a namespace into the current file.
- **using static:** import static members directly (e.g. call **Sin(x)** instead of **Math.Sin(x)**).

- **using Alias = Type:** create a short name for a long type.
- **global using:** applies to every file in the project; place in one file.
- Only one file may have top-level statements.

Access Modifiers

Modifier	Visible to
public	Everyone
private	Same class only (default for members)
protected	Same class + derived classes
internal	Same assembly (default for types)
protected internal	Assembly + derived classes
private protected	Same assembly + derived classes

Variables, Constants, var

```
int n = 100;
double tol = 1.0e-10;

float f = 3.14f; // note f suffix
var x = 2.5; // inferred as double

const double PI = 3.141592653589793;
const int MAX_ITER = 1000;
var list = new List<int>(); // type is List<int>
```

- **const:** compile-time. **readonly:** set once at construction.
- **var** infers the type at compile time; it is not dynamic.

Classes: field Keyword and Properties

```
public class Material(double e, double nu)
{
    // Semi-auto property with 'field' (C# 14)
    :
    public double E
    {
        get => field;
        set => field = value > 0
            ? value : throw new ArgumentException
            ();
    }

    } = e;

    public double Nu
    {
        get => field;
        set => field = value is > -1.0 and < 0.5
            ? value : throw new ArgumentException
            ();
    } = nu;

    public double Density { get; set; } //
    auto
    public double Lambda { get; } //
    read-only
    = e * nu / ((1 + nu) * (1 - 2 * nu));

    public double BulkModulus()
    => E / (3.0 * (1.0 - 2.0 * Nu));
}
```

- **field** (C# 14) accesses the backing field in **get/set**, eliminating manual **_field** declarations.
- Expression-body **=>** for single-expression methods.

Primary Constructors and Initialization

```
public class Node(int id, double x, double y
, double z)
{
    public int Id { get; } = id;
    public double X { get; set; } = x;
    public double Y { get; set; } = y;
    public double Z { get; set; } = z;
}

var n = new Node(5, 0, 0, 0) { X = 1.0, Y =
2.0 };
```

- Primary constructor parameters are in scope for initializers and method bodies.
- Object initializers set properties after construction.

Inheritance and Virtual Methods

```
public abstract class Element(int id, int[]
nodeIds)
{
    public int Id { get; } = id;
    public int[] NodeIds { get; } = nodeIds;
    public abstract double[,] Stiffness();
    public virtual int Order => 1;
}

public class Tri3(int id, int[] n) : Element
(id, n)
{
    public override double[,] Stiffness()
    => throw new NotImplementedException();
    public override int Order => 1;
}
```

- **abstract**: must override; class cannot be instantiated.
- **virtual**: can override. **base** calls the parent.
- Single class inheritance; use interfaces for multiple contracts.

INTERFACES, GENERICS, ENUMS, AND PATTERN MATCHING

Interfaces and Extension Members

```
public interface ISolver
{
    void Solve(ReadOnlySpan<double> rhs, Span<
double> x);
    int Iterations { get; }
}

// A class can implement multiple interfaces
:
public class PCG : ISolver, IDisposable
{
    public void Solve(ReadOnlySpan<double> rhs
,
    Span<double> x) { /* ... */ }
    public int Iterations { get; private set;
    }
    public void Dispose() { /* cleanup */ }
}

// Extension members (C# 14):
extension(ReadOnlySpan<double> span)
{
    public double Norm2()
    {
        double s = 0;
        for (int i = 0; i < span.Length; i++)
            s += span[i] * span[i];
        return Math.Sqrt(s);
    }
}
```

- C# 14 **extension** blocks replace the old **static class** pattern.
- Interfaces can have default implementations since C# 8.

Generics and allows ref struct

```
public class DenseVector<T>(int n)
    where T : INumber<T>
{
    private readonly T[] _data = new T[n];
    public int Length => _data.Length;
    public ref T this[int i] => ref _data[i];

    public T Norm()
    {

```

```
T s = T.Zero;
for (int i = 0; i < _data.Length; i++)
    s += _data[i] * _data[i];
return T.CreateChecked(
    Math.Sqrt(double.CreateChecked(s)));
}
```

```
// allows ref struct (C# 13):
static T Sum<T>(ReadOnlySpan<T> x)
    where T : INumber<T>, allows ref struct
{
    T s = T.Zero;
    for (int i = 0; i < x.Length; i++) s += x[
i];
    return s;
}
```

```
static void Swap<T>(ref T a, ref T b)
=> (a, b) = (b, a);
```

- **allows ref struct** (C# 13) lets generics accept **Span<T>** and other ref-like types.
- JIT compiles specialized versions for each value type.

Enums

```
public enum ElementType
{ Tri3 = 3, Quad4 = 4, Tet4 = 5, Hex8 = 6 }

public enum BoundaryCondition
{ Free, Fixed, Prescribed }

[Flags]
public enum DOF
{
    None = 0, Ux = 1, Uy = 2, Uz = 4,
    Rx = 8, Ry = 16, Rz = 32,
    All = Ux | Uy | Uz | Rx | Ry | Rz,
}
DOF free = DOF.Ux | DOF.Uy;
bool hasUx = (free & DOF.Ux) != 0;
```

Enums are integers underneath. **[Flags]** enables bitwise combination.

Nullable Types and Null Safety

```
// Value types are non-nullable by default:
int n = 5; // cannot be null
int? m = null; // nullable value type

if (m.HasValue) Console.WriteLine(m.Value);
int k = m ?? 0; // null-coalescing
string? name = null;
int len = name?.Length ?? 0; // null-conditional
x ??= new(); // assign only if null
```

- **T?** for value types wraps in **Nullable<T>**.
- **?? / ?. / ??=** for concise null handling.
- Reference types are nullable by default; enable nullable context for warnings.

Pattern Matching and Switch Expressions

```
if (element is Tri3 tri)
    Console.WriteLine(tri.Order);

string name = elemType switch
{
    ElementType.Tri3 => "Triangle",
    ElementType.Quad4 => "Quadrilateral",
    ElementType.Tet4 => "Tetrahedron",
    _ => "Unknown",
};

string Classify(double x) => x switch
{
    < 0 => "negative",
    0 => "zero",
    < 1 => "small",
    >= 1 => "large",
};
```

The discard **_** is the catch-all arm. Property patterns: **{ E: > 200e9 }**.

Tuples, Deconstruction, Records

```
var (det, singular) = ComputeDet(K);

static (double det, bool singular)
    ComputeDet(
        double[, ] K)
{
    double d = /* ... */ 1.0;
    return (d, Math.Abs(d) < 1e-15);
}

// Record: value equality + ToString for free:
public record MaterialProps(double E, double
    Nu, double Rho);

// Record struct: stack-allocated:
public readonly record struct Vec2(double X,
    double Y);
```

- **Tuples**: lightweight multi-return without a dedicated type.
- **record struct**: value semantics + equality + **ToString**.

ARRAYS, INDEXING, AND MULTIDIMENSIONAL DATA

One-Dimensional Arrays

```
double[] x = new double[100]; // zeroed
int[] ids = {0, 1, 2, 3}; // collection expression
x[0] = 1.0;
int n = x.Length;
Array.Resize(ref x, 200); // new array
x.AsSpan(0, 100).CopyTo(y);
Array.Fill(x, 0.0);
Array.Sort(x);
```

- Arrays are heap-allocated reference types. 0-based indexing.
- Bounds checked at runtime (**IndexOutOfRangeException**).
- Collection expressions **[a, b]** work for arrays, lists, spans.

```
// With index, use classic for:
for (int i = 0; i < x.Length; i++) x[i] *= 2.0;

// LINQ: declarative queries (ALLOCATES!)
double sum = x.Sum();
double max = x.Max();
double[] pos = x.Where(v => v > 0).ToArray();
int[] sorted = ids.OrderBy(i => coords[i]).ToArray();
```

- **foreach** cannot modify the collection.
- LINQ allocates iterators, closures, and intermediate arrays.
- **Never** use LINQ inside hot numerical loops.
- LINQ is perfect for setup: mesh preprocessing, filtering, postprocessing.

Multidimensional vs Jagged Arrays

```
double[,] K = new double[3, 3]; // row-major
K[0, 0] = 1.0;
int rows = K.GetLength(0);

double[][] J = new double[3][]; // jagged
J[0] = new double[3];

// Best for performance: flat 1D + manual index
double[] Kf = new double[n * n];
Kf[i * n + j] = value; // row-major
```

- **double[,]**: contiguous block. Jagged **[][]**: scattered rows.
- Flat **double[]** with index arithmetic gives best cache locality.

Array Slicing and Ranges

```
double[] a = [1, 2, 3, 4, 5, 6, 7, 8];
double[] sub = a[2..5]; // ALLOCATES copy!
double last = a[^1]; // 8 (index from end)
```

```
// Span slicing: no copy, no allocation
Span<double> view = a.AsSpan(2, 3);
ReadOnlySpan<double> ro = a.AsSpan()[2..5];
```

- **a[i..j]** on arrays allocates a new array. **AsSpan()** **[i..j]** does not.
- **^n** means “n from the end”.
- Always prefer **Span** slicing in hot paths.

foreach, LINQ, and When Not to Use Them

```
foreach (double xi in x) Console.WriteLine(xi);
```

Operator Overloading

```
public readonly struct Vec3(double x, double y, double z)
{
    public double X { get; } = x;
    public double Y { get; } = y;
    public double Z { get; } = z;

    public static Vec3 operator +(Vec3 a, Vec3 b)
        => new(a.X + b.X, a.Y + b.Y, a.Z + b.Z);
    public static Vec3 operator -(Vec3 a, Vec3 b)
        => new(a.X - b.X, a.Y - b.Y, a.Z - b.Z);
    public static Vec3 operator *(double s, Vec3 v)
        => new(s * v.X, s * v.Y, s * v.Z);
    public static Vec3 operator *(Vec3 v, double s)
        => s * v;
    public static double operator *(Vec3 a, Vec3 b)
        => a.X * b.X + a.Y * b.Y + a.Z * b.Z;

    public double Norm() => Math.Sqrt(this * this);
    public override string ToString()
        => $"({X:G6}, {Y:G6}, {Z:G6})";
}
```

```
// Usage reads like math:
Vec3 a = new(1, 0, 0);
Vec3 b = new(0, 1, 0);
Vec3 c = a + 2.0 * b;
double d = a * b; // dot product
```

Operators must be **static**. Both mixed-type orderings must be defined.

Indexers

```
public class DenseMatrix(int rows, int cols)
{
    private readonly double[] _d = new double[
        rows*cols];
    public int Rows => rows;
    public int Cols => cols;
    public ref double this[int i, int j]
        => ref _d[i * cols + j];
    public ReadOnlySpan<double> Row(int i)
        => _d.AsSpan(i * cols, cols);
}

var K = new DenseMatrix(3, 3);
K[0, 0] = 1.0;
```

Returning **ref double** allows in-place mutation without copying. This is the natural way to wrap flat storage as a 2D matrix. Use **Row()** to get zero-allocation views of individual rows.

EXCEPTION HANDLING, RESOURCES, AND FILE I/O

Exception Handling

```
try
{
    double[] K = LoadStiffness("mesh.dat");
    Solve(K, rhs, x);
}
catch (FileNotFoundException ex)
{
    Console.Error.WriteLine($"Not found: {ex.
        FileName}");
}
catch (ArithmeticException ex)
{
    Console.Error.WriteLine($"Math error: {ex.
        Message}");
}
catch (Exception ex)
{
    Console.Error.WriteLine($"Error: {ex.
        Message}");
    throw; // re-throw preserving stack trace
}
finally { /* always runs: cleanup code */ }
```

- Catch specific types first, general last.
- **throw**; re-throws; **finally** always executes.
- Exceptions in inner loops are expensive; validate beforehand.

IDisposable and using

```
using var reader = new StreamReader("mesh.
    dat");
while (reader.ReadLine() is { } line)
    ProcessLine(line);
// reader.Dispose() called at scope exit

// Custom disposable:
public class ScratchBuffer(int n) :
    IDisposable
{
    private double[][] _buf =
        ArrayPool<double>.Shared.Rent(n);
    public Span<double> Span => _buf.AsSpan();
    public void Dispose()
    {
        if (_buf is not null)
        { ArrayPool<double>.Shared.Return(_buf);
            _buf = null; }
    }
}
```

- **using var**: deterministic cleanup at scope exit.
- Ideal for file handles, pooled buffers, unmanaged resources.

File I/O for Scientific Data

```
string[] lines = File.ReadAllLines("mesh.dat");
List<(int id, double x, double y, double z)>
    nodes = [];
```

```
foreach (string line in lines)
{
    ReadOnlySpan<char> s = line.AsSpan().Trim();
    if (s.IsEmpty || s[0] == '#') continue;
    var p = line.Split(' ',
        StringSplitOptions.RemoveEmptyEntries);
    nodes.Add((int.Parse(p[0]),
        double.Parse(p[1], CultureInfo.
            InvariantCulture),
        double.Parse(p[2], CultureInfo.
            InvariantCulture),
        double.Parse(p[3], CultureInfo.
            InvariantCulture)));
}
```

```
using var sw = new StreamWriter("results.dat");
sw.WriteLine("# Node Ux Uy");
for (int i = 0; i < n; i++)
    sw.WriteLine($"{i} {ux[i]:E14.7} {uy[i]:
        E14.7}");
```

- **StreamReader/StreamWriter** for large-file streaming.
- Always use **CultureInfo.InvariantCulture** for numeric I/O.
- Prefer **TryParse (ReadOnlySpan<char>)** to avoid string allocations.

STRING FORMATTING, DELEGATES, AND CONSOLE OUTPUT

String Interpolation and Formatting

```
string s1 = $"x = {x:F6}"; // 3.141593
string s2 = $"x = {x:E4}"; // 3.1416E
+000
string s3 = $"x = {x:G6}"; // 3.14159
string s4 = $"iter = {iter,6}"; // right-
    aligned
Console.WriteLine($"Step {iter}: res = {res:
    E4}");
```

FORMAT SPECIFIERS

Spec	Meaning	Example
F6	fixed, 6 decimals	3.141593
E4	scientific, 4 dec.	3.1416E+000
G6	general, 6 digits	3.14159
N2	thousands sep.	1,234.56
D5	integer, padded	00042

StringBuilder for Bulk Output

```
var sb = new StringBuilder(capacity: 1024);
sb.AppendLine("# Results");
for (int i = 0; i < n; i++)
{
    sb.Append(i).Append(' ');
    sb.Append(x[i].ToString("E10",
        CultureInfo.InvariantCulture));
    sb.AppendLine();
}
File.WriteAllText("output.dat", sb.ToString());
```

- **string** is immutable; loop concatenation is $O(n^2)$.
- **StringBuilder** amortizes into a growable buffer.
- For peak throughput, use **TryFormat** into span buffers.

Delegates, Lambdas, and Func<>

```
Func<double, double> f = Math.Sin;
Func<double, double> g = x => x * x - 2.0;
double root = Bisect(g, 0.0, 2.0, 1e-12);

static double Bisect(Func<double, double> f,
    double a, double b, double tol)
{
    while (b - a > tol)
    {
        double c = 0.5 * (a + b);

        if (f(a) * f(c) < 0) b = c; else a = c;
    }
    return 0.5 * (a + b);
}
```

- ```
Action<string> log = Console.WriteLine;
```
- **Func<..., TResult>**: returns a value. **Action<...>**: void.
  - Lambdas capture variables (not values); closures allocate.

## MENTAL MODEL AND API SURFACE

## Performance Hierarchy

- Fast code = fewer allocations, fewer cache misses, fewer indirect calls.
- **for** in hot loops; LINQ for orchestration only.
- Copies kill throughput with large structs or frequent slices.
- Measure first: **Stopwatch** for coarse, BenchmarkDotNet for micro.

### RULE OF THUMB

Public API can be rich; internal kernels should be brutally explicit.

## Choose Parameter and Return Types

| Type                                                                                | Use when                        |
|-------------------------------------------------------------------------------------|---------------------------------|
| <code>T[]</code>                                                                    | caller owns concrete array      |
| <code>Span&lt;T&gt;</code>                                                          | mutable window, sync only       |
| <code>ReadOnlySpan&lt;T&gt;</code>                                                  | read-only window, kernels       |
| <code>Memory&lt;T&gt;</code>                                                        | buffer survives async boundary  |
| <code>IReadOnlyList&lt;T&gt;</code>                                                 | indexed public API              |
| <code>IEnumerable&lt;T&gt;</code>                                                   | lazy enumeration, not hot paths |
| <code>ref T / in T</code>                                                           | alias: mutable / read-only      |
| <code>ref readonly T</code>                                                         | return alias without copy       |
| Never expose <code>List&lt;T&gt;</code> unless caller mutation is intended.         |                                 |
| <code>params ReadOnlySpan&lt;T&gt;</code> (C# 13) avoids implicit array allocation. |                                 |

## Struct vs Class

- **struct**: value type. Assignment copies data. Stack or inline.
- **class**: reference type. Assignment copies the pointer. Heap.
- Good struct: tiny, copy-cheap, no identity, no inheritance.
- Bad struct: large matrix, mutable graph node, owns arrays.
- **readonly struct** for tiny immutable values. Pass large structs with **in**.

```
public readonly struct Vec3(double x, double y, double z)
{
 public double X { get; } = x;
 public double Y { get; } = y;
 public double Z { get; } = z;
}

public readonly struct Mat6 // likely too big to copy
{
 private readonly double M00, M01, M02,
 ...;
}
```

## SPANS, MEMORY LAYOUT, AND POOLS

### Span, ReadOnlySpan, stackalloc, scoped

```
Span<double> tmp = stackalloc double[16];
tmp.Clear();

ReadOnlySpan<char> token = line.AsSpan(start, len);
double.TryParse(token, System.Globalization.NumberStyles.Float,
 CultureInfo.InvariantCulture, out double x);

static int CountPositives(scoped ReadOnlySpan<double> x)
{
 int c = 0;
 for (int i = 0; i < x.Length; ++i)
 if (x[i] > 0.0) ++c;
 return c;
}

// params span (C# 13): no hidden array allocation
static double Max(params ReadOnlySpan<double> values)
{
 double m = double.NegativeInfinity;
 foreach (double v in values) if (v > m) m = v;
 return m;
}
```

- **Span<T>**: stack-only, cannot be boxed or cross **await**. Think pointer + length.
- **stackalloc**: tiny scratch buffers. **scoped**: prevents accidental escape.
- Prefer **a.AsSpan(2, 6)** over **a[2..8]** in hot paths (no copy).

### MemoryMarshal and Unsafe

```
Span<byte> bytes = stackalloc byte[64];
Span<double> x = MemoryMarshal.Cast<byte, double>(bytes);
ref double r0 = ref MemoryMarshal.GetReference(x);

unsafe
{
 fixed (double* p = x)
 for (int i = 0; i < x.Length; ++i) p[i] = 0.0;
}
```

- **MemoryMarshal.Cast<A,B>**: only when layout is guaranteed.
- **CollectionsMarshal.AsSpan(list)**: valid only while list is not resized.
- **Unsafe.Add(ref r0, i)** can remove bounds checks, but only when invariants are airtight.
- Prefer explicit layout reasoning over “clever” reinterpretation.
- **unsafe** requires **<AllowUnsafeBlocks>true</AllowUnsafeBlocks>**.

### Pools and Scratch Space

```
var pool = ArrayPool<double>.Shared;
double[] tmp = pool.Rent(n);

try
{
 var span = tmp.AsSpan(0, n);
 span.Clear();
 Work(span);
}
finally
{
 pool.Return(tmp, clearArray: false);
}
```

- Rent when buffers are too large for **stackalloc**.
- Never assume rented arrays are zeroed. Return exactly once.
- For long-lived ownership, allocate your own array instead.
- Think of pooled arrays as pre-allocated work space that avoids GC pressure.

### Low-Level Memory

- Flat arrays beat **T[, ]** and jagged **T[][]** in kernels.
- Zero-initialization is not free; skip it when all values will be overwritten.
- Pinning blocks GC movement; keep pin scopes short.
- Arrays > 85 kB land on the Large Object Heap; prefer **ArrayPool**.
- Locality dominates: contiguous traversal beats random access.
- The GC is generational; short-lived small objects are cheap.

```
static int Idx(int i, int j, int ncols) => i * ncols + j;

// Uninitialized (all values will be overwritten):
double[] buf = GC.AllocateUninitializedArray<double>(n);

// Pinned (won't move during GC):
double[] pin = GC.AllocateArray<double>(n, pinned: true);
```

## COLLECTIONS, APIS, AND HOT-LOOP DISCIPLINE

## Collections in Kernels

```
var list = new List<int>(capacity: n);
list.Add(1);
list.EnsureCapacity(2 * n);

Span<int> s = CollectionsMarshal.AsSpan(list);
for (int i = 0; i < s.Length; ++i) s[i] *= 2;
```

- Pre-size **List<T>** and **Dictionary<K,V>** when cardinality is known.
- TryGetValue** beats contains-then-index. **HashSet<T>** for membership.
- Do not mutate a list after taking its marshal span.

## ESSENTIAL COLLECTIONS

| Type                         | Use case                         |
|------------------------------|----------------------------------|
| <b>List&lt;T&gt;</b>         | Growable array                   |
| <b>Dictionary&lt;K,V&gt;</b> | Hash map (sparse lookups)        |
| <b>HashSet&lt;T&gt;</b>      | Unique membership                |
| <b>Queue&lt;T&gt;</b>        | FIFO buffer (BFS, front advance) |
| <b>Stack&lt;T&gt;</b>        | LIFO buffer (DFS, undo)          |
| <b>SortedDictionary</b>      | Ordered keys                     |

## Dictionary by Reference

```
ref double value = ref CollectionsMarshal
 .GetValueRefOrAddDefault(map, key, out
 bool exists);
if (!exists) value = 0.0;
value += increment;
```

- Avoids repeated hashing and extra lookups.
- The returned ref is invalid if the dictionary resizes.
- Excellent for sparse assembly-like accumulation.

## Span-Based Parsing

```
ReadOnlySpan<char> line = source.AsSpan();
int k = line.IndexOf(';');
ReadOnlySpan<char> left = line[..k];
ReadOnlySpan<char> right = line[k + 1..];
int id = int.Parse(left, CultureInfo.
 InvariantCulture);
double x = double.Parse(right, CultureInfo.
 InvariantCulture);
```

- Always **InvariantCulture** for persisted numeric data.
- SearchValues<char>** and **IndexOfAny** for high-volume tokenization.

## Equality, Hashing, and Stable Keys

- If a value is used as a key, its equality and hash code must be stable for its entire dictionary residency.
- Override **Equals** and **GetHashCode** together.
- Use ordinal string comparison for identifiers: **StringComparer.Ordinal**.
- For floating-point keys, think hard before using raw doubles.

```
public readonly struct Edge(int a, int b)
 : IEquatable<Edge>
{
 public int A { get; } = a;
 public int B { get; } = b;
 public bool Equals(Edge o) => A == o.A &&
 B == o.B;
 public override bool Equals(object? obj)
 => obj is Edge e && Equals(e);
 public override int GetHashCode()
 => GetHashCode.Combine(A, B);
 public static bool operator ==(Edge l,
 Edge r)
 => l.Equals(r);
 public static bool operator !=(Edge l,
 Edge r)
 => !l.Equals(r);
```

```
}
Full contract: IEquatable<T>,
Equals(object?), GetHashCode(),
==/!=. Or just use record struct to
get all for free.
```

## Public API Layering

| Layer                                                                              | Typical signature                                                             |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Convenience                                                                        | <code>double[] Solve(double[] rhs)</code>                                     |
| Core sync                                                                          | <code>void Solve(ReadOnlySpan&lt;double&gt; rhs, Span&lt;double&gt; x)</code> |
| Batched                                                                            | <code>void SolveBatch(ReadOnlySpan&lt;double&gt; A, ...)</code>               |
| Async I/O                                                                          | <code>ValueTask&lt;int&gt; ReadAsync(Memory&lt;byte&gt; dst)</code>           |
| Expose friendly wrappers, but keep one explicit zero-allocation kernel underneath. |                                                                               |

## Closure Trap

```
// BUG: all tasks capture the *same*
variable i
for (int i = 0; i < n; ++i)
 tasks[i] = Task.Run(() => Work(i));

// FIX: copy to a local before capturing
for (int i = 0; i < n; ++i)
{
 int j = i;
 tasks[i] = Task.Run(() => Work(j));
}
```

- Lambdas capture *variables*, not values.
- Closures also cause hidden heap allocations for captured state.

## SIMD, PARALLELISM, AND DIAGNOSTICS

### SIMD and Intrinsics

```
if (Vector.IsHardwareAccelerated)
{
 int w = Vector<double>.Count;
 int i = 0;
 Vector<double> acc = Vector<double>.Zero;
 for (; i <= n - w; i += w)
 acc += new Vector<double>(a, i) * new Vector<double>(b, i);
 double s = 0.0;
 for (int j = 0; j < w; ++j) s += acc[j];
 for (; i < n; ++i) s += a[i] * b[i];
}
```

- Start with **Vector<T>** before AVX intrinsics.
- Vector128<T>** / **Vector256<T>** for explicit width control.
- Data layout often matters more than instruction cleverness.

### Parallel Loops

```
Parallel.For(0, n, i => y[i] = F(x[i]));

Lock sync = new(); // System.Threading.Lock (C# 13)
double total = 0.0;
Parallel.For<double>(0, n,
 () => 0.0,
 (i, _, local) => local + a[i],
 local => { lock (sync) total += local; });

var opts = new ParallelOptions
{ MaxDegreeOfParallelism = Environment.ProcessorCount };
Parallel.For(0, nElem, opts, e =>
{
 Span<double> Ke = stackalloc double[ndof * ndof];
 ComputeElement(e, Ke);
 lock (sync) AssembleGlobal(e, Ke, K);
});
```

- Lock** (C# 13) replaces **object-as-lock** with a purpose-built,

more efficient type.

- Parallelize only when per-iteration work is large enough.
- Avoid shared writes; partition by output slices.
- Beware false sharing when adjacent elements are written by different threads.
- Deterministic scientific code usually wants explicit partitioning.

### Diagnostics and Measurement

```
Span<char> dst = stackalloc char[64];
if (value.TryFormat(dst, out int written,
 provider: CultureInfo.InvariantCulture))
 Console.WriteLine(dst[..written].ToString());
```

```
var sw = Stopwatch.StartNew();
for (int i = 0; i < nReps; i++) Solve(K, rhs, x);
sw.Stop();
Console.WriteLine(
 $"Avg: {sw.Elapsed.TotalMilliseconds / nReps:F2} ms");
```

- Warm up before timing. Release build only. No debugger.
- Track allocations as a first-class metric, not only elapsed time.
- Tools: **dotnet-counters**, **dotnet-trace**, BenchmarkDotNet.

### BENCHMARK CHECKLIST

- Stable input, pinned CPU frequency if possible, multiple iterations, verify outputs.